CHAPTER 8 IMPLEMENTING THE DATABASE WITH SQL SERVER AND INTRODUCTION TO DEVOPS & DEPLOYMENT STRATEGIES (PART 1/2)

Week 8 is where the project moves from temporary in-memory data storage to a real, persistent database. In previous weeks students built requirements, user stories, UML models, user interfaces, CRUD logic, and basic MVC organization. By Week 8 the course outline expects students to implement the actual database of the system using SQL Server. The lecture topic is DevOps & deployment strategies (1/2), the lab focuses on problem solving implementing the database using SQL Server, the assignment is Assignment #5 (Implementing Database of System using SQL Server), and students also complete Personal Journal 4. This week therefore combines two important goals: moving the project to a professional database backend and beginning to think about how the system will be deployed and maintained in a real environment.

8.1 WHY WEEK 8 IS A MAJOR MILESTONE

Up to now the project has used a temporary in-memory List to store data. This approach was useful for quick prototyping and testing CRUD operations in Weeks 6 and 7. However, an in-memory List has serious limitations: data disappears when the program closes, it cannot support multiple users, and it is not scalable. Week 8 solves these problems by replacing the temporary storage with a real relational database using Microsoft SQL Server.

This change is significant because it transforms the project from a simple prototype into a system that behaves like real-world information systems.

8.2 WHAT IS SQL SERVER AND WHY USE IT IN THIS COURSE?

SQL Server is Microsoft's relational database management system. It is widely used in industry, integrates extremely well with C#, and supports all the concepts students learned in Week 6 (tables, primary keys, foreign keys, relationships, normalization, and CRUD operations).

In this course SQL Server is the chosen database technology because:

- It works natively with C# and .NET
- It supports the exact database design students created in Week 6
- It prepares students for real-world development environments

8.3 MOVING FROM IN-MEMORY LIST TO SQL SERVER

The transition from the temporary List<Appointment> (or similar collection) used in Lab 6 and Assignment #4 to a real SQL Server database is one of the most important technical steps in the entire course.

TECHNICAL LIMITATIONS OF THE IN MEMORY LIST APPROACH

In previous weeks the data was stored in code like this:

C#

```
private List<Appointment> appointments = new List<Appointment>();

// This line declares a private field that holds a dynamic list of Appointment objects in memory.
```

This List exists only in the computer's RAM while the program is running. As soon as the application is closed, all data is lost. Additionally, it has no built-in support for:

- Data persistence across multiple runs
- Concurrent access by multiple users
- Complex querying and filtering
- Data integrity rules (foreign keys, constraints)
- Security and backup features

WHY SQL SERVER SOLVES THESE PROBLEMS

SQL Server stores data permanently on disk (or in the cloud). Once a record is inserted, it remains available even if the program is restarted or the computer is turned off. It also provides powerful querying capabilities through SQL, enforces relationships via foreign keys, and supports proper data validation and security.

TECHNICAL TRANSITION PROCESS

The move from List to SQL Server involves these concrete technical steps:

1. **Create the actual database and tables in SQL Server** Students open SQL Server Management Studio (or Azure Data Studio) and create a new database (e.g., AdvisingSystemDB). Then they define tables that match their Week 6 design, including primary keys, foreign keys, appropriate data types, and constraints.
2. **Add a connection string in the C# project** In App.config or appsettings.json (or directly in code), students add a connection string that tells the application how to reach the database.
3. **Use ADO.NET to connect and execute SQL commands** The most common classes are SqlConnection, SqlCommand, and SqlDataReader/SqlDataAdapter.
4. **Replace List-based CRUD with SQL-based CRUD** Instead of appointments.Add(...), students now execute an INSERT statement. Update and Delete use UPDATE and DELETE statements with parameters.

DETAILED TECHNICAL BENEFIT

After this transition, when a student submits an appointment request, the data is no longer stored only in RAM, it is physically saved in SQL Server files. This makes the system persistent, more secure, and ready for future multi-user or web-based deployment.

8.4 CREATING THE DATABASE AND TABLES IN SQL SERVER

Students will create a new database (for example, AdvisingSystemDB) and define the tables based on their Week 6 design.

Practical SQL Example: Creating the Database and Tables

SQL

```
CREATE DATABASE AdvisingSystemDB;

-- Creates a new database with the specified name. All tables will be stored inside this database.

GO

USE AdvisingSystemDB;

-- Switches the current context to the newly created database so that subsequent commands affect it.

GO

CREATE TABLE Students

(

   studentID INT IDENTITY(1,1) PRIMARY KEY,

   -- IDENTITY(1,1) automatically generates a unique number starting at 1. PRIMARY KEY ensures uniqueness and indexing.

   name NVARCHAR(100) NOT NULL,

   -- NVARCHAR allows Unicode characters (e.g., accented names). NOT NULL means the column cannot be empty.

   email NVARCHAR(150) NOT NULL,

   program NVARCHAR(100) NOT NULL

);

CREATE TABLE Advisors

(

   advisorID INT IDENTITY(1,1) PRIMARY KEY,

   name NVARCHAR(100) NOT NULL,

   department NVARCHAR(100) NOT NULL

);
```

```
CREATE TABLE Appointments
(
   appointmentID INT IDENTITY(1,1) PRIMARY KEY,
   studentID INT NOT NULL,
   advisorID INT NOT NULL,
   [date] DATETIME NOT NULL,
   -- Square brackets are used because "date" is a reserved word in SQL Server.
   reason NVARCHAR(255) NOT NULL,
   status NVARCHAR(50) NOT NULL,
   CONSTRAINT FK_Appointments_Students
      FOREIGN KEY (studentID) REFERENCES Students(studentID),
      -- Creates a relationship that ensures every studentID in Appointments must exist in Students.
   CONSTRAINT FK_Appointments_Advisors
      FOREIGN KEY (advisorID) REFERENCES Advisors(advisorID)
);
```

SEED DATA EXAMPLE

SQL

```
INSERT INTO Students (name, email, program)
VALUES
('Pejman Azadi', 'pejman@example.com', 'Computer Science'),
('Sara Lee', 'sara@example.com', 'Software Engineering');
-- Inserts two sample students so the system has test data immediately after table creation.


INSERT INTO Advisors (name, department)
VALUES
('Dr. Chen', 'Computer Science'),
('Dr. Smith', 'Engineering');
```

Example The Appointments table will have:

- appointmentID as the primary key (auto-increment)
- studentID as a foreign key referencing Students.studentID
- advisorID as a foreign key referencing Advisors.advisorID

This structure enforces referential integrity and prevents invalid data.

8.5 CONNECTING C# TO SQL SERVER (ADO.NET BASICS)

C# uses ADO.NET to communicate with SQL Server. The most common classes are:

- SqlConnection: opens a connection to the database
- SqlCommand: executes SQL statements
- SqlDataReader: reads data returned from SELECT queries

PRACTICAL C# CONNECTION EXAMPLE

C#

```
using System.Data.SqlClient;

// Imports the namespace that contains all SQL Server-related classes so we can use SqlConnection, SqlCommand, etc.


string connectionString = @"Server=(localdb)\MSSQLLocalDB;Database=AdvisingSystemDB;Integrated Security=True;";

// This string tells the program exactly which database server and database to connect to. Integrated Security=True uses Windows login.

// Note: The variable "connectionString" is assumed to be already declared as a class-level field or constant in the form (or passed into the method).


try

{

   using (SqlConnection conn = new SqlConnection(connectionString))

   // Creates a new connection object. "using" ensures the connection is automatically closed when finished.

   {

      conn.Open();

      // Actually opens the connection to SQL Server. If this fails, an exception is thrown.

      MessageBox.Show("Database connection successful.");

   }
```

```
}

catch (Exception ex)

{

   MessageBox.Show("Database error: " + ex.Message);

   // Catches any connection problems (wrong server name, database doesn't exist, etc.) and shows a user-friendly message.

}
```

Students will add a connection string that tells the application how to reach the SQL Server database.

NOTE ON EXECUTENONQUERY VS EXECUTEREADER

- ExecuteNonQuery() is used for INSERT, UPDATE, and DELETE (returns the number of affected rows).
- ExecuteReader() is used when you need to read rows from a query result.
- SqlDataAdapter is convenient when filling a DataTable for a DataGridView.

8.6 IMPLEMENTING CRUD WITH SQL SERVER

Week 8 is the point where students replace the in-memory List CRUD code with real SQL statements.

Practical SQL Examples

Create (INSERT)

SQL

```
INSERT INTO Appointments (studentID, advisorID, [date], reason, status)

VALUES (1, 2, '2025-10-20 10:00:00', 'Course planning discussion', 'Requested');

-- Inserts one new row into the Appointments table. The values must match the column order and data types.
```

Read with JOIN

SQL

```
SELECT

   a.appointmentID,              -- Selects the unique ID of each appointment from the Appointments table

   s.name AS StudentName,        -- Selects the student's name from the Students table and renames the column for clarity

   adv.name AS AdvisorName,      -- Selects the advisor's name from the Advisors table and renames the column for clarity

   a.[date],                     -- Selects the appointment date (brackets used because "date" is a reserved word)

   a.reason,                     -- Selects the reason text entered by the student

   a.status                      -- Selects the current status of the appointment (Requested, Approved, etc.)
```

```
FROM Appointments a                -- Starts from the Appointments table and gives it the alias "a"
INNER JOIN Students s              -- Joins the Students table (alias "s") so we can get student names
   ON a.studentID = s.studentID    -- Matches appointments to students using the foreign key studentID
INNER JOIN Advisors adv            -- Joins the Advisors table (alias "adv") so we can get advisor names
   ON a.advisorID = adv.advisorID  -- Matches appointments to advisors using the foreign key advisorID
ORDER BY a.[date] ASC;             -- Sorts the final result by appointment date in ascending order (earliest first)
```

Update

SQL

```
UPDATE Appointments
SET status = 'Approved'
WHERE appointmentID = 5;
-- Changes the status of a specific appointment. The WHERE clause is critical to avoid updating every row in the table.
```

Delete

SQL

```
DELETE FROM Appointments
WHERE appointmentID = 5;
-- Removes one specific appointment. Again, WHERE prevents accidental deletion of all records.
```

Note about reserved words

The word date is a reserved word in SQL Server, so it is enclosed in square brackets [date] to avoid errors.

FULL C# INSERT EXAMPLE

C#

```
string query = "INSERT INTO Appointments (studentID, advisorID, [date], reason, status) " +
               "VALUES (@studentID, @advisorID, @date, @reason, @status)";
// Builds the SQL command as a string. Uses parameters (@...) instead of directly inserting values.


using (SqlConnection conn = new SqlConnection(connectionString))
using (SqlCommand cmd = new SqlCommand(query, conn))
```

```
{
    conn.Open();                        // Opens the database connection so the INSERT can be executed
    cmd.Parameters.AddWithValue("@studentID", 1);    // Safely passes the studentID value to the first parameter
    cmd.Parameters.AddWithValue("@advisorID", 2);    // Safely passes the advisorID value to the second parameter
    cmd.Parameters.AddWithValue("@date", DateTime.Now); // Safely passes the current date and time to the date parameter
    cmd.Parameters.AddWithValue("@reason", txtReason.Text); // Safely passes the text from the Reason textbox to the reason parameter
    cmd.Parameters.AddWithValue("@status", "Requested"); // Safely passes the literal string "Requested" to the status parameter

    cmd.ExecuteNonQuery();  // Executes the INSERT command and returns how many rows were affected
}
```

FULL C# UPDATE EXAMPLE

C#

```
string query = "UPDATE Appointments SET status = @status WHERE appointmentID = @appointmentID";
// Prepares an UPDATE statement using parameters for security.

using (SqlConnection conn = new SqlConnection(connectionString))
using (SqlCommand cmd = new SqlCommand(query, conn))
{
    conn.Open();                        // Opens the database connection so the UPDATE can run
    cmd.Parameters.AddWithValue("@status", "Approved");     // Adds the new status value safely via parameter
    cmd.Parameters.AddWithValue("@appointmentID", 5);       // Adds the ID of the record to update safely via parameter
    cmd.ExecuteNonQuery();                  // Executes the UPDATE command and returns how many rows were changed
}
```

FULL C# DELETE EXAMPLE

C#

```
string query = "DELETE FROM Appointments WHERE appointmentID = @appointmentID";
// Prepares a DELETE statement using a parameter.


using (SqlConnection conn = new SqlConnection(connectionString))
using (SqlCommand cmd = new SqlCommand(query, conn))
{
   conn.Open();                          // Opens the database connection so the DELETE can run
   cmd.Parameters.AddWithValue("@appointmentID", 5);      // Adds the ID of the record to delete safely via parameter
   cmd.ExecuteNonQuery();                    // Executes the DELETE command and returns how many rows were removed
}
```

FULL C# READ EXAMPLE (LOADING INTO DATAGRIDVIEW)

C#

```
// Note: You also need "using System.Data;" at the top of the file because DataTable is from the System.Data namespace.
private void LoadAppointments()
{
   string query = "SELECT appointmentID, studentID, advisorID, [date], reason, status FROM Appointments";
   // Simple SELECT query to retrieve all appointment records.


   using (SqlDataAdapter adapter = new SqlDataAdapter(query, connectionString))
   // SqlDataAdapter is designed to fill a DataTable from a SELECT query.
   {
      DataTable dt = new DataTable();
      adapter.Fill(dt);          // Executes the query and fills the DataTable with the results
      dgvAppointments.DataSource = dt; // Binds the DataTable directly to the DataGridView so records appear on screen
   }
}
```

8.7 INTRODUCTION TO DEVOPS AND DEPLOYMENT STRATEGIES (PART 1/2)

DevOps is a set of practices that combines software development (Dev) and IT operations (Ops) to shorten the development life cycle and deliver high-quality software continuously.

In this course, Week 8 introduces the first half of DevOps and deployment thinking. Key concepts include:

- Version control (Git) for tracking changes
- Building and testing the application
- Preparing the application for deployment (moving from development to a production-like environment)

Technical DevOps Note

Keep SQL scripts and connection strings in version control. Store connection strings in configuration files rather than hard-coding them. Separate development and production configurations to make deployment safer and repeatable.

8.8 WHY DEPLOYMENT MATTERS NOW

By Week 8 students have a working system with a real database. The next logical question is: "How will this system actually run for real users?" Deployment strategies answer that question. Even in a student project, thinking about deployment helps students write more professional and maintainable code.

8.9 CONNECTING WEEK 8 TO PREVIOUS CHAPTERS

Week 8 builds directly on everything that came before:

- Week 6 database design becomes the actual SQL Server tables
- Week 7 MVC structure organizes the new database code
- CRUD logic from Week 6 is now made permanent and integrated into the more organized project structure developed by Week 7
- Deliverable II (Week 7) showed improved organization, Week 8 makes the data layer professional

8.10 WHAT STUDENTS SHOULD KNOW BY THE END OF CHAPTER 8

By the end of this chapter, students should be able to demonstrate the following knowledge and skills:

- **Explain the difference between in-memory storage and a real SQL Server database** Students must clearly describe why an in-memory List is useful for early prototyping (fast, no server needed) but insufficient for real applications (data is lost when the program closes, not scalable, no multi-user support). They should also explain the advantages of SQL Server (persistent storage, data integrity, security, scalability).
- **Create basic tables in SQL Server that match their Week 6 design** Students must be able to design and create tables with appropriate primary keys, foreign keys, data types, and relationships that reflect their entity-relationship diagram from Week 6.
- **Write simple C# code to connect to SQL Server and perform CRUD operations** Students must understand how to use SqlConnection, SqlCommand, and basic ADO.NET techniques to execute INSERT, SELECT, UPDATE, and DELETE statements from their Windows Forms application.
- **Understand the basic ideas of DevOps and deployment strategies** Students must explain what DevOps is, why it matters in modern system development, and the first concepts of deployment (version control, building the application, moving from development to production environments).

- **Prepare the database implementation required for Assignment #5 and Personal Journal 4** Students must be ready to implement a working database for their project and reflect on the transition from temporary storage to a persistent database.

8.11 CHAPTER SUMMARY

Chapter 8 moves the project from temporary in-memory data to a real, persistent SQL Server database. Students learn how to create tables, connect C# to the database, and replace List-based CRUD operations with SQL-based operations. At the same time, the chapter introduces the first part of DevOps and deployment strategies, helping students think about how their system will eventually be built, tested, and delivered. By the end of Week 8 the project has a solid, professional data foundation and is much closer to a real-world information system.

REFERENCES

- Vanier College, Faculty of Continuing Education. (2025). *Course Outline: System Development (420-940-VA)*. Vanier College.
- Shelly, G. B., Cashman, T. J., & Rosenblatt, H. J. (2014). *System Analysis and Design* (10th ed.). Course Technology. ISBN: 9781285171340.
- Kendall, K. E., & Kendall, J. E. (2023). *Systems Analysis and Design*. Pearson Education. ISBN-13: 9780137947850.
- Microsoft Learn. (2025–2026). *SQL Server and ADO.NET documentation*. Microsoft Learn.
- Freeman, A., & Robson, E. (2022). *Pro C# 10 with .NET 6* (11th ed.). Apress. (Chapter on database access with ADO.NET and Entity Framework)
- Kim, G., Humble, J., Debois, P., & Willis, J. (2016). *The DevOps Handbook*. IT Revolution Press.